

Binary tree → count the no. of nodes.
Height of Node.

traversal → any traversal (B.T)

operation
for, while

preorder
inorder
postorder.

void preorderTraversal(Node)

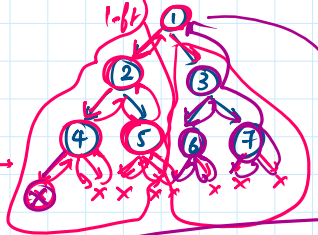
7 = nodes
6 = edges.

if (root == NULL) {
return;

1
cout << root->data; [Node]
preorderTraversal(root->left) [left]
preorderTraversal(root->right) [right]

Node
preorder:
preordering

1 2 3
Node, left, right.
print, storing → print



preorder
o/p: 1 2, 4, 5, 3, 6, 7



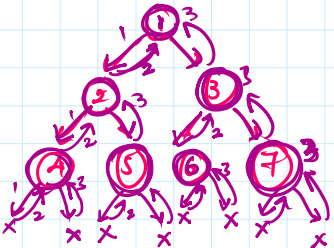
inorder
1 2 3
left, Node, right
print, storing

inorder.
o/p: 4 2, 5, 1, 6, 3, 7

preorder: → Node, left, right
inorder: → left, Node, right
postorder: → left, right, Node

post traversal

o/p: 4, 5, 2, 6, 7, 3, 1



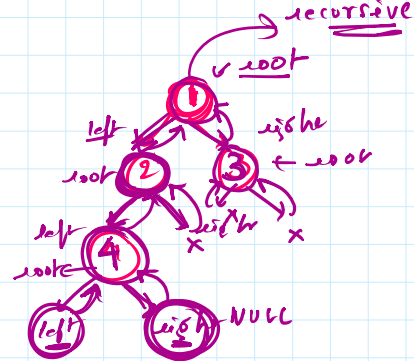
1 2 3
left, right, Node

print, answer,
inorder: left, Node, right

o/p 4, 2, 5, 1, 3

void inorderTraversal(Node, answer)
if (root == NULL) {
return;
}

left → inorderTraversal(root->left, answer);
Node → answer.push_back(Node);
right → inorderTraversal(root->right, answer);



o/p 4 2 1 3



Inorder Traversal Solved

Difficulty: Easy Accuracy: 67.03% Submissions: 12384 Points: 3 Average Time: 1.0s

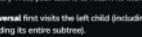
★

Given a **root** of a Binary Tree, your task is to return its **Inorder Traversal**.

Note: An **inorder traversal** first visits the left child (including its entire subtree), then visits the node, and finally visits the right child (including its entire subtree).

Examples:

Input: root = [1, 2, 3, 4, 5]



```

graph TD
    1((1)) --> 2((2))
    1 --> 3((3))
    2 --> 4((4))
    2 --> 5((5))
    
```

Output: [4, 2, 5, 1, 3]
Explanation: The inorder traversal of the given binary tree is [4, 2, 5, 1, 3].

Input: root = [8, 3, 5, 6, 7, 10, 6, 8, 10, 6]



```

graph TD
    8((8)) --> 3((3))
    8 --> 5((5))
    3 --> 6((6))
    3 --> 7((7))
    5 --> 10((10))
    5 --> 6((6))
    
```

```

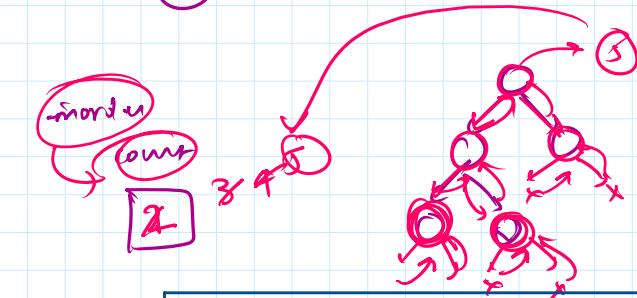
1 // Inorder Traversal
2 public:
3     void InorderTraversal(Node* root, vector<int>&ans){
4         if(!root) return;
5         InorderTraversal(root->left, ans);
6         ans.push_back(root->data);
7         InorderTraversal(root->right, ans);
8     }
9     vector<int> Inorder(Node* root) {
10        vector<int> ans;
11        InorderTraversal(root, ans);
12        return ans;
13    }

```

```

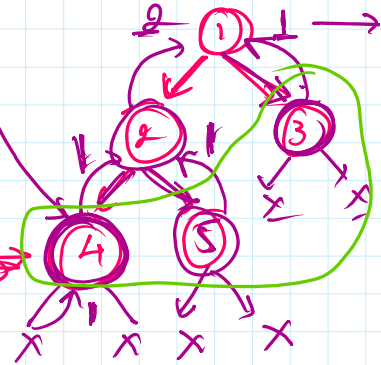
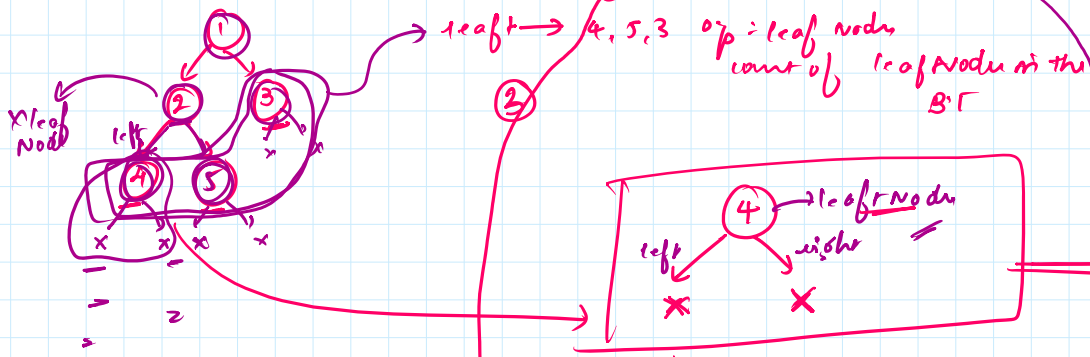
1 // Inorder Traversal
2 public:
3     void InorderTraversal(Node* root, vector<int>&ans){
4         if(!root) return;
5         InorderTraversal(root->left, ans);
6         ans.push_back(root->data);
7         InorderTraversal(root->right, ans);
8     }
9     vector<int> Inorder(Node* root) {
10        vector<int> ans;
11        InorderTraversal(root, ans);
12        return ans;
13    }

```





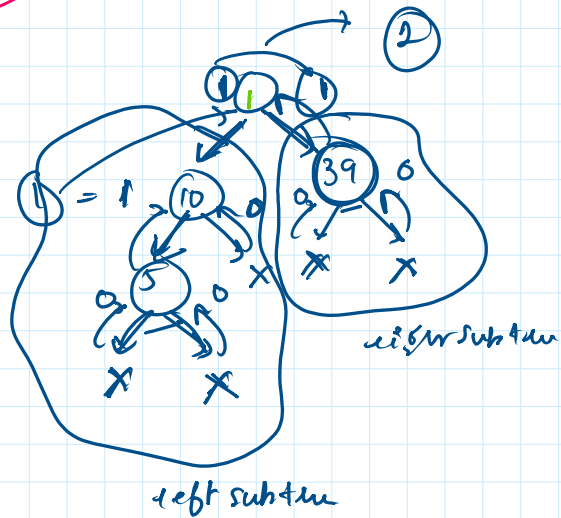
count the number of leaf nodes $\rightarrow$ (left & right == NULL) $\rightarrow$ count++



leaf = 5139

count = 2

```
void countLeaf(____, ____){
    if (root == NULL)
        return 0;
    if (root->left == NULL & root->right == NULL)
        return 1;
    return countLeaf(root->left) + countLeaf(root->right);
}
```



③ com g

0

2